EXPERIMENT NO. 01—Network Generation and Spanning Tree on Network



Name: Dibyanshu Shekhar Dey
Roll Number: 25EC64R23

Design and Simulation Lab: Communication Networks
Department of Electronics & Electrical Communication Engineering
Indian Institute of Technology Kharagpur

AIM: Network Generation and Spanning Tree on Network

1. Adjacency and cost matrix view of network
2. Density gradient
3. Minimum weight Spanning Tree and Prim's Algorithm

INTRODUCTION:

A graph $G = (V, E)$ can be represented using an adjacency matrix A of size $|V| \times |V|$, where $a_{ij} = 1$ if there is an edge between vertices i and j, and $a_{ij} = 0$ otherwise [1]. In an undirected graph, the adjacency matrix is symmetric about its main diagonal. Let the transpose of a matrix $A = (a_{ij})$ be defined as $A^T = (a^T_{ij})$, where $a^T_{ij} = a_{ji}$. Since edges (u, v) and (v, u) represent the same connection in an undirected graph, the adjacency matrix satisfies $A = A^T$. This symmetry enables memory optimization by storing only the diagonal and upper-triangular entries of the matrix [1].

The adjacency-matrix representation can be extended to weighted graphs by storing the edge weight $w(u, v)$ as the matrix entry corresponding to row u and column v. If no edge exists between two vertices, the corresponding entry may store a NIL value or a convenient placeholder such as 0 or ∞, depending on the application [1].

Let us consider the following graph.

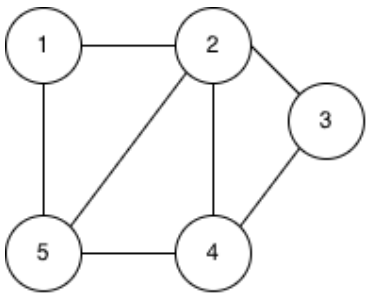


Figure 1: Undirected non-weighted graph [1]

This is an undirected graph which is unweighted. Its corresponding adjacency matrix is given below.

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

As one can see, if two vertices are not connected, its representation in matrix form is NIL, whereas if connected, its representation is represented by 1. Also note the symmetry in the matrix along the diagonal. Note also the 0 along the diagonal elements - which means that not a single node is connected with itself.

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Figure 2: Matrix showing the symmetry around diagonals and vertices connected and non-connected [1]

1. Yellow Line - showing the diagonal elements are zero and matrix is symmetric over this diagonal
2. Red Line - showing symmetry along right to left diagonal
3. Green circle - showing the connected vertices between vertex 5 and vertex 1
4. Blue square - Vertices that are not connected between vertex 3 and vertex 1

THEORY:

Centralized Systems: Centralized systems have all or most of the processing done on a single central server. This helps in managing all the operations through a central server that acts as as hub trhough which all the requests are processed. Because of single point of control, the architecture is simple, easy to deploy and manage and uses resources efficiently. However there is limited scalability and if the central server fails, the entire server will be inoperable [2].

Prim's algorithm is based on Centralized Systems.

Decentralized Systems: Decentralized systems are computing architectures in which control and processing are distributed across multiple independent nodes rather than managed by a central authority. By allowing nodes to operate autonomously while coordinating with each other, such systems improve fault tolerance, scalability, and resilience. The absence of a single point of failure, along with distributed communication and redundancy, enables reliable operation even in the presence of node failures or attacks [2].

Distributed Systems: Distributed systems consist of multiple independent computers that communicate and coordinate over a network to function as a single unified system. By enabling resource sharing, concurrent execution, and transparency to users, distributed systems improve performance, scalability, and reliability while maintaining fault tolerance through redundancy and replication [2].

Graph Density: For an undirected simple graph, the graph density is defined as

$$D = \frac{2|E|}{|V|(|V|-1)}.$$

For a directed simple graph, the graph density is defined as

$$D = \frac{|E|}{|V|(|V|-1)}.$$

Here, $|E|$ denotes the number of edges and $|V|$ denotes the number of vertices in the graph [3]. The maximum possible number of edges in an undirected simple graph is given by

$$\frac{|V|(|V|-1)}{2}.$$

Random Graph: A random graph is a graph made by connecting vertices (nodes) randomly. This means the connections between the nodes are determined randomly, based on certain rules or probabilities. Random graphs are used to study networks where the exact connections are not known beforehand, but follow some random pattern [4].

Spanning Tree and Minimum Spanning Tree : A spanning tree of a connected, undirected graph is a subgraph that connects all vertices without forming cycles. Since a graph may have multiple spanning trees, weights can be assigned to edges to represent their cost. The weight of a spanning tree is defined as the sum of the weights of its edges. A minimum spanning tree (MST) is a spanning tree whose total weight is less than or equal to that of any other spanning tree in the graph [5].

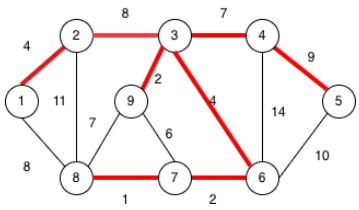


Figure 3: Minimum Spanning Tree (MST) with edge weights shown and edges in MST in red. Total weight = 37 [1]

Properties of MST: Following are some of the properties of MST:

1. Possible multiplicity: There can be multiple MST for a graph, with all of them having same minimum weights [5].
2. Uniqueness: Given a graph has all the weights unique, then MST formed is also unique [5].
3. Minimum cost subgraph: If the edges of a graph have non-negative weight, then MST is minimum cost subgraph [5].

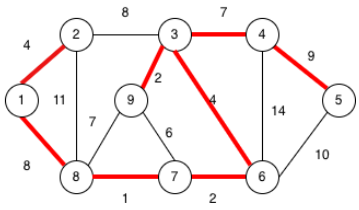


Figure 4: Minimum Spanning Tree (MST) with edge weights shown and edges in MST in red. Total weight = 37. Note that the MST is non unique for this graph (see the figure above). Here nodes 2-3 replaced by 1-8. [1]

Minimum Spanning Trees (MSTs) are widely used in the design of cost-efficient networks. They help determine the minimum-cost connections without cycles in applications such as communication networks, airline route optimization, and infrastructure planning involving cities, electronic components, or computer systems. MSTs are also employed in routing algorithms to identify efficient network paths [5].

Prim's Algorithm: Prim's algorithm is a greedy algorithm used for MST for a connected weighted undirected graph. There are 3 kinds of vertices that are used in Prim's algorithm [4], [5].

1. Tree Vertices: Vertices that are part of MST T.
2. Fringe Vertices: Vertices that are currently not part of T, but are adjacent to some tree vertex.
3. Unseen vertices: Vertices that are neither part of tree nor fringe vertices.

Algorithm 1 Prim's Algorithm for Minimum Spanning Tree

Select an arbitrary starting vertex s
Initialize the tree $T \leftarrow \{s\}$
while there exist fringe vertices **do**
    Select an edge (u, v) such that $u \in T$, $v \notin T$
    Choose (u, v) with minimum weight
    Add vertex v to T
    Add edge (u, v) to the minimum spanning tree
end while
EXIT

RESULTS:

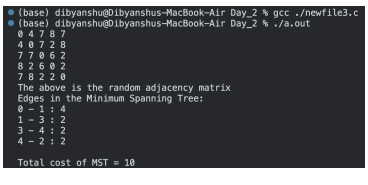


Figure 5: Result in the terminal showing the random graph with 5 vertices

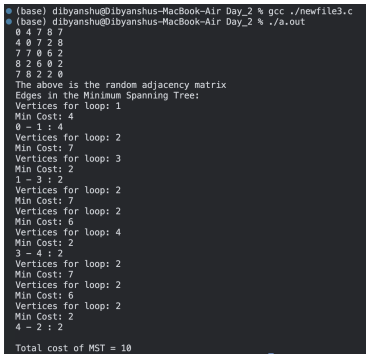


Figure 6: Result in the terminal showing the random graph with 5 vertices. This time the print statements are used to generated loop traversal and choose of minimum weights for 5 vertices graph

Output for 5 Vertex Graph

```
0 4 7 8 7
4 0 7 2 8
7 7 0 6 2
8 2 6 0 2
7 8 2 2 0
The above is the random adjacency matrix
Edges in the Minimum Spanning Tree:
0 - 1 : 4
1 - 3 : 2
3 - 4 : 2
4 - 2 : 2

Total cost of MST = 10
```

Output for 10 Vertex Graph

```
0 7 7 9 8 7 2 3 3 6
7 0 8 6 1 5 9 10 1 4
7 8 0 1 10 9 6 4 6 9
9 6 1 0 2 1 9 6 5 1
8 1 10 2 0 1 11 5 10 10
7 5 9 1 1 0 8 4 8 2
2 9 6 9 11 8 0 3 7 5
3 10 4 6 5 4 3 0 8 2
3 1 6 5 10 8 7 8 0 3
6 4 9 1 10 2 5 2 3 0
The above is the random adjacency matrix
Edges in the Minimum Spanning Tree:
0 - 6 : 2
0 - 7 : 3
7 - 9 : 2
9 - 3 : 1
3 - 2 : 1
3 - 5 : 1
```

```
5 - 4 : 1
4 - 1 : 1
1 - 8 : 1

Total cost of MST = 13
```

DISCUSSION:
The adjacency matrix created is as follows:

	0	1	2	3	4
0	0	4	7	8	7
1	4	0	7	2	8
2	7	7	0	6	2
3	8	2	6	0	2
4	7	8	2	2	0

In the following steps, the graph for 5 vertices were created. These are pictorial representation of step by step transition to find MST. All the vertices are uninitiated.

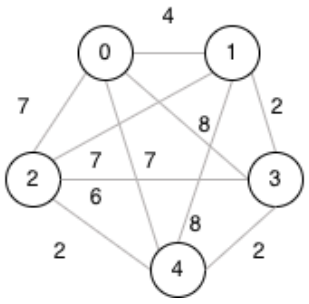


Figure 7: Completed graph for 5 Vertices

Vertex 0 is selected, so in set T, 0 is added. Now 0 will search through all the edges outgoing (see in blue).

Set T
$T = \{0\}$

Outgoing edges from 0 along with its weights.
$(0, 1) \rightarrow 4$
$(0, 3) \rightarrow 8$
$(0, 4) \rightarrow 7$
$(0, 2) \rightarrow 7$

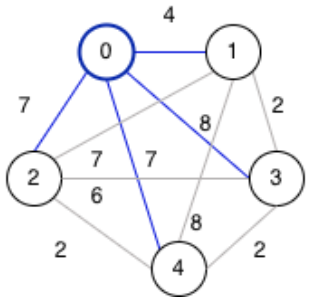


Figure 8: Vertex 0 searches through the edges. Set T has $\{0\}$.

The one with the minimum weight is selected i.e $(0, 1) \rightarrow 4$.

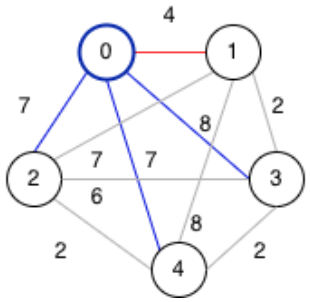


Figure 9: $(0, 1) \rightarrow 4$ selected for MST.

1 is added to T, resulting in $T = \{0\ , 1\}$. Outgoing edges now under consideration, before 1 starts searching are
$(0, 3) \rightarrow 8$
$(0, 4) \rightarrow 7$
$(0, 2) \rightarrow 7$

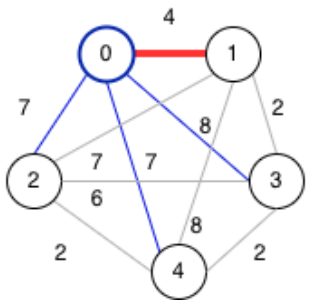


Figure 10: MST with 0 and 1.

Now 1 will start seached for its outgoing edges. The edges under consideration along with the nodes are:
$(0, 3) \rightarrow 8$
$(0, 4) \rightarrow 7$
$(0, 2) \rightarrow 7$
$(1, 3) \rightarrow 2$

$(1, 4) \rightarrow 8$
$(1, 2) \rightarrow 7$

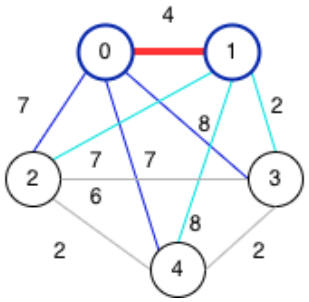


Figure 11: 1 searching its outgoing edges. The algorithm will now consider all the edges still under consideration.

The minimum edge that selects the fringe vertex is of weight 2. This is from 1 to vertex 3. The edges under consideration are:
$(0, 3) \rightarrow 8$
$(0, 4) \rightarrow 7$
$(0, 2) \rightarrow 7$
$(1, 4) \rightarrow 8$
$(1, 2) \rightarrow 7$

Set T will now have 3. The updated set is $T = \{0 , 1 , 3\}$

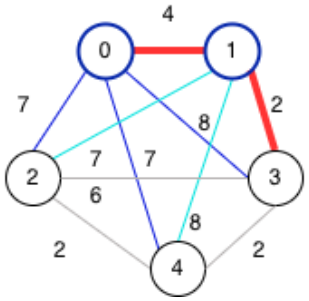


Figure 12: MST formed with vertices 0, 1, 3

Now vertex 3 will search through all its vertices and algorithm will check if a node has not been updated in set T and which edge has minimum edge. Few of the edges are already under consideration, so they will be as they are.

Edges under consideration now are:
$(0, 3) \rightarrow 8$
$(0, 4) \rightarrow 7$
$(0, 2) \rightarrow 7$
$(1, 4) \rightarrow 8$
$(1, 2) \rightarrow 7$
$(3, 4) \rightarrow 2$

$(3, 2) \rightarrow 7$

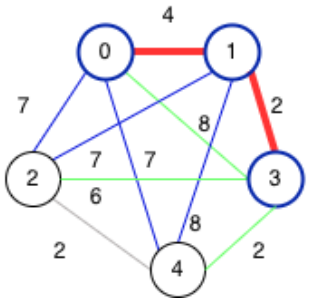


Figure 13: Vertex 3 adding the edges into consideration.

$(3, 4) \rightarrow 2$ has the minimum weight out of all the edges under consideration. Also 4 is not in set T. So 4 will be added to the set T.

Edges under consideration now are:
$(0, 3) \rightarrow 8$
$(0, 4) \rightarrow 7$
$(0, 2) \rightarrow 7$
$(1, 4) \rightarrow 8$
$(1, 2) \rightarrow 7$
$(3, 2) \rightarrow 7$

The updated set T is $T = \{0 , 1 , 3 , 4\}$.

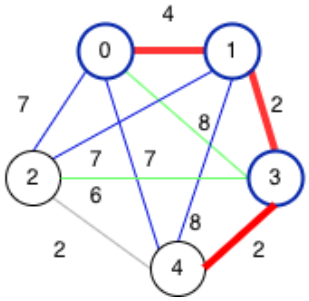


Figure 14: Updated MST with Vertex 4.

Now 4 will go through all the edges. Some of the edges are already under consideration.
Edges under consideration now are:
$(0, 3) \rightarrow 8$
$(0, 4) \rightarrow 7$
$(0, 2) \rightarrow 7$
$(1, 4) \rightarrow 8$
$(1, 2) \rightarrow 7$
$(3, 2) \rightarrow 7$
$(4, 2) \rightarrow 2$

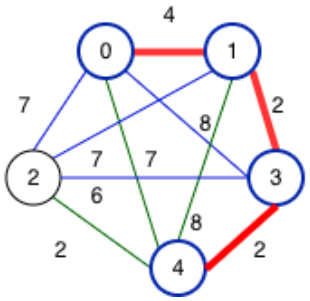


Figure 15: Vertex 4 checking its edges and all the edges under consideration.

Now the minimum edge is selected $(4, 2) \rightarrow 2$. All the edges have been visited along with all the vertices. The updated set T is $T = \{0\ ,1\ ,3\ ,4\ ,2\}$.

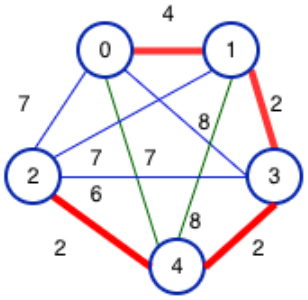


Figure 16: Minimum weight edge is selected that connects 2 and 4, which completes the MST.

The final completed MST is as follows:

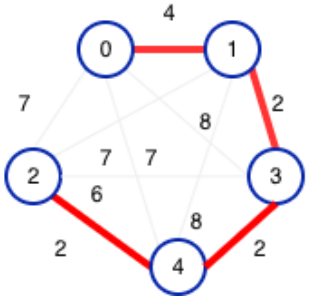


Figure 17: Final MST

The minimum weight is 10.

CONCLUSION:
Prim's Algorithm was used on a randomly generated adjacency matrix using C. The future work on this will include the user's response for degree (between 0 and 1) and the number of nodes so that a random generated adjacency matrix can be created from user's input without user knowing what is the weight of the edge.

ACKNOWLEDGEMENT:
I would like to share my thanks to Prof. Raja Datta for setting the stage of usage of such algorithms. I would share my thanks to Prof. Amitalok J Budkuley for elaborating on Prim's

algorithm. I would like to extend my thanks to TA's Anuj Meena, E.Maansi, Ajay Mehra, Pradeep Reddy, Mudumbai Venkata Srivatsav, Prince Kumar, Baisnabi Rout and Siva Sankar Thummala for questioning the fundamentals of the experiment and helping throughout. I would like to thank my colleagues Mayank Singh, Saptarshi Bhattacharjee, Soumava Biswas and Soumyajit Adak for collaborating in lab throughout the experiment.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd. Cambridge, MA, USA: MIT Press, 2009.

[2] GeeksforGeeks, *Comparison of centralized, decentralized and distributed systems*, GeeksforGeeks, Accessed: 2026-01-18, 2026. [Online]. Available: https://www.geeksforgeeks.org/system-design/comparison-centralized-decentralized-and-distributed-systems/.

[3] J. A. B. Dris, *What is the definition of the density of a graph?* Mathematics Stack Exchange, Accessed: 2026-01-18, Nov. 2015. [Online]. Available: https://math.stackexchange.com/q/1526421.

[4] Tutorialspoint, *Graph theory – random graphs*, Online Tutorial, Accessed: 2026-01-18, n.d. [Online]. Available: https://www.tutorialspoint.com/graph_theory/graph_theory_random_graphs.htm.

[5] R. Thareja, *Data Structures Using C*, 2nd. New Delhi, India: Oxford University Press, 2014.

APPENDIX (CODE):

```c
#include<stdio.h>
#include <stdlib.h>

#define R 5
# define upper 11
# define lower 1


void addEdge(int mat[R][R], int i, int j){
    int a = rand() % (upper - lower + 1) + lower;
    if(i == j){
        mat[i][j] = 0;
        mat[j][i] = 0;
    }
    else{
        mat[i][j] = a;
        mat[j][i] = a;
    }

}
void displayMatrix(int mat[R][R]){
    // looping through the matrix to display it
    for (int i = 0; i < R; i++)
    { // loops through row
        for (int j = 0; j < R; j++){
```

```
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
}

void primMST(int mat[R][R]) {
    int visited[R] = {0};
    int edgeCount = 0;
    int minCost = 0;

    visited[0] = 1;

    printf("\nEdges in the Minimum Spanning Tree:\n");

    while (edgeCount < R - 1) {
        int min = 9999;
        int u = -1, v = -1;

        for (int i = 0; i < R; i++) {
            if (visited[i]) {
                for (int j = 0; j < R; j++) {
                    if (!visited[j] && mat[i][j] != 0 && mat[i][j] < min) {
                        printf("Vertices for loop: %d\n", j);
                        min = mat[i][j];
                        printf("Min Cost: %d\n", min);
                        u = i;
                        v = j;

                    }
                }
            }
        }

        visited[v] = 1;
        edgeCount++;
        minCost += min;

        printf("%d - %d : %d\n", u, v, min);
    }

    printf("\nTotal cost of MST = %d\n", minCost);
}


int main(){

    int mat[R][R] = {0};

    for (int i = 0; i < R; i++){
        for(int j = 0; j < R; j++){
            addEdge(mat, i, j);
        }
    }

    displayMatrix(mat);
    printf("The above is the random adjacency matrix");

```

```
84      primMST(mat);
85
86      return 0;
87  }
```